

# Propagating Visual Designs to Numerous Plots and Dashboards

Saiful Khan<sup>1\*</sup>, Phong H. Nguyen<sup>2†</sup>, Alfie Abdul-Rahman<sup>3‡</sup>, Benjamin Bach<sup>4§</sup>, Min Chen<sup>1¶</sup>,  
Euan Freeman<sup>5||</sup>, Cagatay Turkey<sup>6\*\*</sup>

<sup>1</sup> University of Oxford, <sup>2</sup> Redsift Ltd., <sup>3</sup> King's College London, <sup>4</sup> Edinburgh University, <sup>5</sup> University of Glasgow, <sup>6</sup> University of Warwick

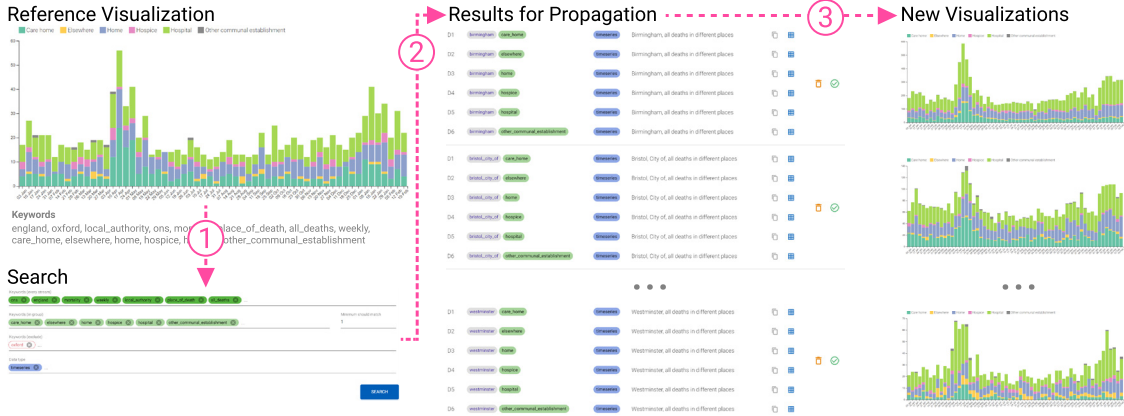


Fig. 1. Our novel propagation workflow makes it easy to propagate visual designs to numerous datasets. Reference visualizations are created for data streams, which are associated with several keywords in our ontology. A search and activate process is used to propagate the reference visualization to other appropriate data streams. (1) Ontology keywords are used to construct a query in our search UI for suitable data stream combinations. (2) Search results consist of ranked data stream combinations that match query parameters, although some results may not be suitable for propagation. (3) A quality assurance step carried out by an expert ensures the visual design is only propagated to suitable data, resulting in new visualizations that are immediately deployed as web pages.

**Abstract**—In the process of developing an infrastructure for providing visualization and visual analytics (VIS) tools to epidemiologists and modeling scientists, we encountered a technical challenge for applying a number of visual designs to numerous datasets rapidly and reliably with limited development resources. In this paper, we present a technical solution to address this challenge. Operationally, we separate the tasks of data management, visual designs, and plots and dashboard deployment in order to streamline the development workflow. Technically, we utilize: an ontology to bring datasets, visual designs, and deployable plots and dashboards under the same management framework; multi-criteria search and ranking algorithms for discovering potential datasets that match a visual design; and a purposely-design user interface for propagating each visual design to appropriate datasets (often in tens and hundreds) and quality-assuring the propagation before the deployment. This technical solution has been used in the development of the RAMPVIS infrastructure for supporting a consortium of epidemiologists and modeling scientists through visualization.

**Index Terms**—Visualization system, propagation, infrastructure, ontology, quality assurance, pandemic, emergency response.

## 1 INTRODUCTION

RAMPVIS [8] is a group of volunteers specialized in Visualization and Visual Analytics (VIS), who answered a call to support the modeling scientists and epidemiologists in the Scottish COVID-19 Response Consortium (SCRC). One major challenge identified at the beginning (May 2020) was that there was a huge amount of data that epidemiologists and modeling scientists in the SCRC needed to access rapidly but could only do so via data files in a variety of inconsistent formats, requiring time-consuming processing. In most cases, they had to create simple plots using spreadsheet facilities, as they lacked the expertise and tools

to create visual designs tailored to their needs. Meanwhile, another group of volunteers in the SCRC started developing a data infrastructure for hosting captured data, as well as computational data resulting from model testing, uncertainty analysis, and model predication. A subset of VIS volunteers formed a *generic support team* focusing on providing a VIS infrastructure coupled with the SCRC data infrastructure, while other VIS volunteers gathered detailed domain-specific requirements for analytical, model-developmental, and disseminative visualization, and developed techniques for supporting these visualization needs.

Although the generic support team managed to bring together a large cohort of UK-based VIS volunteers who could contribute to the design and engineering of a VIS system for visualizing thousands of datasets, the total programming resources in the team was less than one full-time person. The team thus streamlined their development effort by componentizing different tasks in the workflow, from obtaining data streams from the SCRC data infrastructure, to making visualization facilities available to the end users. By July 2020, the team produced simple plots and dashboards, and could semi-automatically propagate them to numerous data streams of similar data types (e.g., propagating a plot across data from all regions of a country). In this context, propagation means binding a visualization function to numerous data streams to deploy new interactive visualizations (as in Fig. 1). This helped shape

\*e-mail: saiful.khan@eng.ox.ac.uk

†e-mail: phong.nguyen@redsift.io

‡e-mail: alfie.abdulrahman@kcl.ac.uk

§e-mail: bbach@ed.ac.uk

¶e-mail: min.chen@oerc.ox.ac.uk

||e-mail: euan.freeman@glasgow.ac.uk

\*\*e-mail: cagatay.turkay@warwick.ac.uk

the VIS infrastructure through rapid development with minimal cost.

However, since then, many more datasets have become available and more complex plots and dashboards have been added to the VIS infrastructure. The process of propagating these to new data streams becomes more complex, requiring more advanced support and quality assurance. While existing systems (e.g., Tableau or Power BI) provide powerful user interfaces for creating plots and dashboards, they do not provide a propagation mechanism for transforming the design of a plot or dashboard with multiple data sources to many hundreds of plots or dashboards with similar but less-well-defined data sources in an efficient, scalable, quality-assured manner.

For example, as illustrated in Fig. 1, a stack bar chart was developed by a VIS volunteer for juxtaposing six time series representing the fatalities in different location types (e.g., care home, hospital) in a geographic region. As over 300 regions in the UK have similar time series data, another VIS volunteer, a dedicated *infrastructure manager*, used the existing stack bar chart as a template, searched for all possible sets of six time series that match with the template, checked for errors in the search results, and finally activated the propagation individually or in groups. The result was several new interactive visualizations covering all regions, created, deployed, and linked to other visualizations with minimal development time and cost. In this paper, we present the development of the VIS infrastructure that enables this cost-effective process for propagating visual designs to numerous plots and dashboards.

Our main contributions include:

- A novel design of an **ontology-based infrastructure** for enabling search for matching data sets;
- A **streamlined workflow** that helps deploy the limited programming resources cost-effectively;
- A **user interface** designed for the infrastructure managers to perform propagation operations;

Perhaps most importantly, we hope that our approach can be utilized and adapted in future VIS efforts in emergency situations.

## 2 RELATED WORK

### 2.1 Automatic Visualization

Zhu et al. [41] presented a review on automatic tools and systems for generating visualizations. In their review, they divided the tools and systems into four different types: (i) tools that require programming and visualization knowledge, such as D3.js [5] and Vega-Lite [30]; (ii) tools that utilize a visual building step, e.g., Charticulator [28] and Lyra [29]; (iii) systems and tools that are semi-automated and require some form of user interaction for generating visualizations, such as Voyager [36] and Show Me [26]; and (iv) automatic visualization generation tools and systems that are designed for users who are not experts in programming or visualization, e.g., Text-to-Viz [11] and Click2Annotate [10]. We extend this classification, contributing a novel infrastructure and visual design pipeline that takes a visualization created by a designer and semi-automatically propagates it across a large data infrastructure.

Brodie et al. [6] presented a survey on a range of visualization applications requiring infrastructural support. Building on the emergence of autonomic computing as a new research agenda at that time [19], they envisaged the need for introducing adaptive and autonomic techniques for managing VIS infrastructures. They started a discussion around infrastructure requirements for such systems, which we contribute to here through our infrastructure design and algorithmic support for visualization. Grammel et al. [16] presented a short survey of visualization construction user interfaces systems dividing the systems into six different approaches of: visual builder, visualization spreadsheet, textual programming, visual dataflow programming, template editor, and shelf configuration. Our propagation pipeline approach is a novel extension of the “visual dataflow programming” approach in their classification.

Several tools for generating visualizations automatically have been explored in the literature, of which we give a brief overview. Mackinlay [25] presented one of the first tools that automatically generates visualizations of relational information, such as bar charts, scatter plots, and connected graphs. Their approach is ideal for domains with easily defined semantics, although is impractical for a problem domain as complex and ever-changing as the response to a pandemic. Mackinlay

et al. [26] described Show Me, user interface commands integrated into Tableau that provide automatic views during the visual design workflow, part of a user-centric approach to visualization generation.

Falconer et al. [14] presented an approach for generating customized visualizations through ontology mapping. Sun et al. [33] demonstrated Articulate, a novel conversational approach to visualization generation. Their system combined natural language processing and machine learning methods to enable the translation of imprecise sentences provided by the user into explicit expressions, which then automatically create a visualization through a heuristic graph generation algorithm. This aimed to simplify the visualization process by allowing users to describe what they wanted to see, without needing to know how to implement the visualization themselves. Cui et al. [11] also explored a natural language approach to visualization generation; they demonstrated an automatic approach for generating infographics from natural language statements – the statements are converted from simple proportion-related statistics to infographics using pre-designed styles.

Automatic visualization approaches can be extended beyond single visualizations to more narrative forms of information dissemination. Shi et al. [32] presented Calliope, a system for automatically generating visual data stories from a spreadsheet. Their system progressively generates story points using a Monte-Carlo tree search algorithm, then assembles these into a single data story. Tang et al. [34] described PlotThread, an AI-assisted system for designing storyline visualizations. Their system provides an AI agent that works alongside the user to collaboratively produce a visual design, one of many examples of AI-assisted visualization generation systems [37].

Users are generally trusting such (semi-)automatically produced visualizations [40]. Using such systems can lower the entry costs to visualization by simplifying the design process for the end user, or by taking them out of the loop entirely. However, as we will discuss in Section 3, a new approach was needed when creating a VIS infrastructure where bespoke visualizations (e.g., for epidemiologists and modeling scientists) needed to be rapidly deployed across a significant data infrastructure in a fast and cost-effective way.

### 2.2 Ontology-Supported Visualization

Ontologies can be a powerful tool in a data infrastructure, supporting the creation of visualizations through their structured representation of data, concepts, and relations [9]. Ontologies and their encoded knowledge may also need to be visualized and there are many techniques for doing this. In this paper we focus on the use of an ontology to support visual design, rather than visualizing an ontology. For an overview of the latter, see surveys by Katifori et al. [18] and Dudáš et al. [12].

Carpendale et al. [7] presented a viewpoint on ontologies in biological data visualization, taking inspiration from their widespread use in biology research. They reflected on the technical challenges of ontology-based visualization in this domain and identified promising future research directions for the visualization community. Among these were ontology-supported visualization generation, leveraging the structure of an ontology to simplify visual design and exploration.

Ontology-supported automatic visualization was explored by Gilson et al. [15], who presented an pipeline approach that combined ontology mapping and probabilistic reasoning to produce new visualizations. Their SemViz system exemplified this process, using three ontologies to automatically visualize music chart data. Khan et al. [21] used an ontology in an enterprise search system to capture search provenance, using the ontology to visualize collaborative search graphs.

Yu and Silva [38] presented VisFlow, a visualization framework where a data flow diagram is used to support exploration and visualization of tabular data. Whilst this work did not use an ontology, its use of structured data representation for visualization is relevant and shows the value in using such representations to support the creation of new visualizations. Their FlowSense system [39] extends this with a natural language interface for editing the data flow diagrams. Its semantic parser with special utterance tagging and placeholders are used to allow generalization to different datasets and data flow diagrams, simplifying visualization creation for the end user.

Our work uses an ontology to support the creation of new visualizations. We leverage the ontology in our data infrastructure to support propagating visual designs across many datasets, through a search-and-review workflow. Our streamlined workflow reduces the time-cost and volunteer effort necessary to scale the RAMPVIS system and support its domain experts with new visual designs.

### 3 PROBLEM STATEMENT AND SYSTEM OVERVIEW

When the VIS volunteers first joined the SCRC effort for combating COVID-19 in May 2020, the SCRC data infrastructure was under development. From some example datasets in spreadsheets, we were overwhelmed by the amount of data. There were time series for different regions, genders, age groups, key indicators (e.g., number of tests, number of ICU patients, etc.), fatality locations (i.e., care homes, hospitals, etc.). There were different models being developed and tested, each of which would produce several time series for different transition states, and hundreds of multiples of such time series for uncertain or sensitivity analysis. Meanwhile, analytical algorithms, e.g., for comparing different datasets, were expected to result in even more datasets, potentially in a combinatorial manner. The scale of such an operation was already significant and would only grow over time.

Although the RAMPVIS generic support team consists of all VIS volunteers who offered to help engineer VIS systems and support others in the SCRC, there was only a very limited amount of programming resource: four VIS volunteers, about 2–15 hours per week per person, totaling 20–30 person-hours per week. Due to the nature of volunteering, we also had to assume that some VIS volunteers might become unavailable from time to time. This required us to devise a highly cost-effective approach, technically as well as operationally. We considered several optional approaches:

- *Using an existing platform that would allow us to create plots and dashboards without programming.* 🚫 We could not use this approach because (i) the generic support team had to implement novel and nuanced visual designs produced by other teams [8]; (ii) we did not have any funds to purchase a server license and consultancy for database connection; and (iii) creating and managing numerous plots and dashboards would be challenging.
- *Programming plots and dashboards with a UI for browsing suitable data.* 🚫 We did not use this approach because (i) it burdens domain experts with browsing hundreds or thousands of data streams; (ii) it burdens each VIS volunteer with knowing all relevant data streams, programming interaction, and being responsible for the full data and visualization pipeline; and (iii) it would demand substantial and consistent availability of resources.
- *Programming with an advanced development framework (freeware).* 🚫 We tried this approach for two weeks but stopped because (i) we realized that only one person was knowledgeable about the suggested framework and libraries, and the learning curve for other volunteers was too high; and (ii) we had doubts about how this would scale to numerous plots and dashboards.
- *Programming reference plots and dashboards using a familiar platform and developing an infrastructure to propagate reference visual designs to work with all similar datasets.* 👍 After some discussion, we took this approach because (i) each developer needed to cover a narrow spectrum of software development, facilitating a streamlined workflow; (ii) because all developers knew D3.js, development could start immediately without the burden of retraining or raising funds; and (iii) it reduces the development time for producing hundreds of plots and dashboards, ideal for a time-critical volunteer effort.

Within two months the team developed several plots and dashboards and were able to propagate all plots with a single data stream. While this rapid development helped convince domain experts in the SCRC to make VIS the fourth pillar in combating COVID-19 (in addition to data, models, and policies), it also encouraged our team to develop more advanced propagation methods for more complex plots and dashboards.

Fig. 2 gives an overview of the main VIS infrastructure components and illustrates the overall workflow, from obtaining data, to creating plots and dashboards, to propagating these across all datasets and making them available to the domain experts. There are three main op-

erations in the workflow overseen by VIS volunteers with distinct roles: *data manager*, *visualization developer*, and *infrastructure manager*:

1. **Obtaining data streams—Fig. 2 (1):** When a new **data product** needs to be visualized, a *data manager* writes a manifest to obtain **data streams**, assigns appropriate keywords, and enters metadata into the ontology via a simple web form.
2. **Writing VIS functions—Fig. 2 (2):** A **VIS function** is an implementation that visualizes data streams in a single web page, e.g., as a plot or dashboard. When domain experts require new designs, a *visualization developer* is given a code template, implements the visual design and binds with reference data streams, creating a reference visualization accessible as a new web page.
3. **Propagating to other streams—Fig. 2 (3):** An *infrastructure manager* uses our search UI to find data similar to the reference data in a VIS function then activates propagation for appropriate results, propagating that design across numerous data streams. These operations that are performed by VIS volunteers are supported by key technical components of the infrastructure:
  - a. **Download agent—Fig. 2 (a):** Periodically obtains data streams from the SCRC data infrastructure without human input.
  - b. **Ontology—Fig. 2 (b):** A knowledge representation of data streams, VIS functions, and their combinations as web pages. Supports search activities during propagation and general use for supporting the domain experts' activities.
  - c. **Web-based visualization service—Fig. 2 (c):** Hosts numerous visualization plots and dashboards in a scalable manner.
  - d. **Algorithmic support—Fig. 2 (d):** Supports propagation by searching for similar data streams registered in the ontology, ranking similarity levels, and ordering the search results.
  - e. **Propagation service—Fig. 2 (e):** Propagation is supported by a specialized UI for searching and activating propagation.

Components (a)–(c) are essential parts of the VIS infrastructure, but were enhanced to support components (d) and (e) which were specially developed for propagating visualizations. All components have been developed as open-source software and the code is available [20,22,23]. The design and development of (d) and (e) took 200~400 person-hours.

### 3.1 VIS Infrastructure Ontology

In our VIS infrastructure, an ontology is used to organize data and visualizations, and to support propagation. We argue that an ontology is a suitable method to deal with the sheer number of diverse data streams and visualizations that we have in our problem context. Considering also that the set of data and designs are constantly evolving over time, a well-designed ontology not only forms the basis of the infrastructure design but also provides versatile and robust means to manage the propagation process. Fig. 3 shows a schema of our ontology. *OntoData*, *OntoVis*, and *OntoPage* are the three main classes, representing data streams, VIS functions and web pages, respectively (as in Fig. 2). The ontology is implemented as a graph data structure: objects are mapped to nodes, relationships between objects are mapped to edges, and directed edges distinguish start/end nodes. In the following sections, we discuss data streams (Section 4), visualization functions (Section 5), and our novel propagation process for binding these to create new visualizations (Sections 6 and 7).

### 4 DATA STREAM

Data streams are units of data in our infrastructure, with associated keywords and metadata. Our infrastructure provides access to data streams via a RESTful API. The *OntoData* class in our ontology stores registered data streams and their attributes (see Fig. 3). From these attributes, *endpoint*, *description*, and *keywords* are most relevant: the endpoint is an RESTful API endpoint used for accessing the data; the description should describe the data and is used for search and discovery; and the keywords describe the stream contents, and are used for searching, grouping, and propagation.

We create data streams using data from the SCRC infrastructure [31], which includes a wide range of COVID-19 data. SCRC data is organized into data products (e.g., testing, hospital, mortality), each further divided into components (e.g., deaths per council area, deaths per age



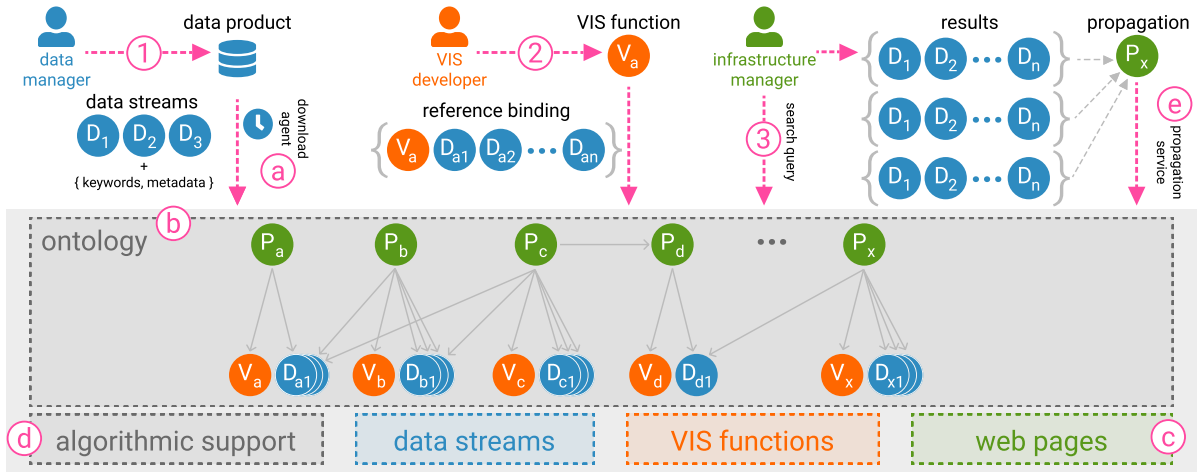


Fig. 2. An illustration of our streamlined workflow, data infrastructure and VIS volunteer roles. (1) When new **data products** become available, a developer writes a manifest to extract **data streams**, add them to the **ontology (b)**, and keep data up to date via a **download agent (a)** that periodically queries the data product for new data. (2) When a new visualization is needed, a developer creates a **visualization function** (e.g., in D3.js) using a template and binds it to reference data stream(s) in the ontology; this new visual design is accessible as a web page, visualizing the reference data. (3) To propagate the reference visualization to other related data streams, the infrastructure manager uses a search UI to find suitable data streams then performs quality assurance on search results, produced by **algorithmic support (d)**. When a decision is made to propagate a visualization function to data stream(s), a **propagation service (e)** operates using the ontology. Propagated visualizations are immediately published as interactive **web pages (c)** for domain experts. The supplementary video walks through this workflow.

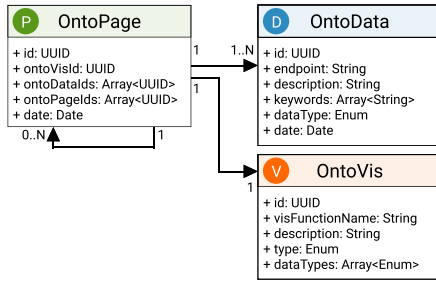


Fig. 3. Schema showing the core classes in our ontology and their relationships to each other.

group). To fetch data, a VIS volunteer writes a download agent to extract data from the SCRC infrastructure and transform it if needed (e.g., to normalize per 100,000 people). Relevant keywords are assigned, then the stream is registered in our ontology. Data products are updated periodically by SCRC, so download agents update data daily.

Data streams can be searched using the OntoData description and keyword attributes, in  $O(n)$  time. To improve efficiency, the Lucene [4] open-source text search engine is used to create an inverted index [27] that maps description and keywords attributes to their matching instances. The inverted index is a hash map-based data structure, allowing searches in  $O(1)$  time complexity. An indexing agent periodically scans the ontology database logs for changes and keeps the index updated.

Description fields are broken into individual words, 2-grams and 3-grams for indexing. The words and their  $n$ -grams support partial matching in a search and allow hints while typing queries, simplifying infrastructure manager operations. For example, a description with “positive cases” would have all its components indexed: “positive” and “cases”; a query for either word would return this string. Keywords are indexed as-is and are not broken down for partial matching.

## 5 VISUALIZATION FUNCTIONS

### 5.1 Visual Design Workflow

In our infrastructure, a VIS function is an implementation of a visual design (e.g., plots, dashboards). These functions are created by a VIS volunteer using familiar libraries (in this instance, D3.js [1]). The OntoVis class in our ontology stores visualization functions and their attributes (see Fig. 3). From these attributes, *visFunctionName* is most important: this is an identifier for a Javascript function that will create

an interactive visualization for given data streams.

A visualization function will be linked to a set of data streams and rendered on a web page for domain experts and visualization viewers. The OntoPage class in our ontology represents a web page, establishing a link between one VIS function (an OntoVis instance) and a set of data streams (one to many OntoData instances), as shown in Fig. 3. Note that each OntoPage instance may also be linked to other OntoPage instances; for example, a dashboard may show several plots and be linked to their individual OntoPage instances.

When a VIS developer needs to implement a new visual design (e.g., to support domain expert requirements), they liaise with the infrastructure manager, who will: (1) create an OntoVis instance, registering the function in the ontology; then (2) create an OntoPage instance, by binding the new OntoVis instance to an appropriate set of reference data streams (i.e., instances of OntoData). The new OntoPage instance results in a web page ‘template’ with placeholder code that the VIS developer can use to implement the visual design. The reference data serves two purposes: providing test data for the developer to support implementation, and providing an initial binding between the VIS function and data streams in the ontology.

Our visual design workflow is based on the core concept that visualization implementation is decoupled from the rest of the infrastructure. When the infrastructure manager creates the reference ‘template’ in the ontology, it appears in the development instance; VIS developers then implement their VIS function and push their code to the repository, making it available immediately. This streamlines the process because developers do not need to know about, or work directly with, the underlying data infrastructure. This makes our approach suited for volunteering operations. It also facilitates efficient propagation for producing numerous plots and dashboards with minimized time-cost.

### 5.2 Creating Web Pages

Each OntoPage instance in our ontology yields an interactive web page that domain experts can use to access a plot or dashboard. Our implementation uses the Flask Jinja template engine [2] to extract information from an OntoPage instance and generate a web page:

1. Title and description are extracted from OntoPage attributes;
2. HTTP requests fetch data from each OntoData API endpoint;
3. The visualization function is identified from the OntoVis instance and its JavaScript object is retrieved from an object factory;
4. The visualization function is called with the fetched data streams, rendering the visualization on the page;

5. If the OntoPage is linked to other OntoPages, these attributes create hyperlinks to their web pages.

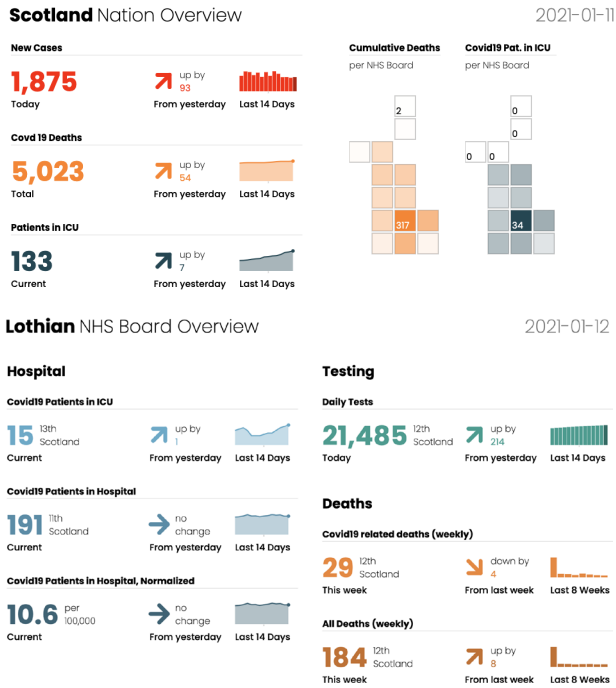


Fig. 4. Example dashboards for Scotland and a regional health board. The nation cartogram links to dashboards for the health board regions.

### 5.3 Template-Based VIS Function Design

Our template-based approach to visualization implementation supports a variety of visual designs, which we categorize as *plots* (single visualizations) or *dashboards* (bespoke composite visual designs with multiple plots, annotations, etc). We briefly discuss how these are implemented in our infrastructure.

**Plots**—We implement a variety of visualizations (e.g., line chart, bar chart, area chart, chord diagram, matrix, map) for many different data types (e.g., time series, cumulative time series, matrix, geographic data). Our infrastructure and propagation process is agnostic to the detailed visualization design and implementation, so is able to accommodate all of the domain experts' visualization needs.

Plots can visualize multiple data streams, as illustrated in Fig. 1. In that example, a stacked bar chart shows weekly location of death in a region of England, with a unique data stream for each of the six locations. The relationships between the VIS function and its six data streams are in the ontology (via the OntoPage instance). By implementing plots in this way, a single VIS function can be propagated across hundreds of data streams. As shown in Fig. 1, we created this plot with reference data streams from Oxford (left plot), then propagated that single function to all regions in England (e.g., Birmingham, City of Bristol, and Westminster in the right plots). When this propagation occurs, each data stream needs to be replaced by the appropriate data stream for the other regions (e.g., replacing Oxford deaths in hospital with Birmingham deaths in hospital).

Plots may also have links to other plots (OntoPage→OntoPage). For example, a plot showing small multiples of COVID-19 patients in ICU of all national health boards can link to each health board.

**Dashboards**—As well as individual plots, our approach is also capable of supporting composite dashboards with several complementary plots drawing from different data streams. Dashboards summarize important data about multiple data streams, such as current data and trends from recent days. Dashboards serve the following purposes: (i) they provide quick access to frequently-used plots; (ii) they provide rapid access to critical information to inform daily decision making (e.g., deciding to call an emergency meeting, or checking if model

predictions match current data); and (iii) they avoid unnecessary search activities, simplifying decision making and review processes.

Fig. 4 shows two of our implemented dashboard visualizations. These summarize data from all of Scotland (top) and one region of Scotland (bottom). Each dashboard has been carefully designed to give an immediate and accurate overview over relevant data, satisfying domain expert requirements. Importantly, each component in a dashboard is linked to the corresponding web page for the individual plot: a viewer can click any of the numbers, arrows, or trend charts to open the full detail view. The cartogram in the nation overview (top) shows each of the NHS Scotland health boards; each region in the cartogram is linked to the dashboard for that health board region, so that clicking a region will lead to regional dashboard, e.g., NHS Lothian in Fig. 4 (bottom).

In total we designed five dashboards each centering about a specific topic such as: a particular region a nation in the UK, hospitals, schools, places of death. Using our propagation mechanism we can propagate these dashboard designs to all Scottish regions, ensuring that each data stream is replaced by the appropriate data stream for the other region.

Dashboards are implemented using the same process as individual plots. Each dashboard has an OntoPage instance, a single VIS function that produces the visual design and page layout, and a set of all associated data streams. Each component in the dashboard is linked to its individual visualization web page: e.g., New Cases in the Nation Overview is linked to the web page visualizing daily cases and the cartogram regions in Nation Overview are linked to the web page for the regional dashboards. These links are stored in the OntoPage attributes and are linked by the VIS function. Consequently, propagating dashboards is more complex than propagating simple visualizations, as the data streams and links need to be correctly matched.

## 6 PROPAGATION SERVICE

In our ontology (Fig. 3), OntoPage objects create a binding between visualization functions (OntoVis), data streams (OntoData) and other web page links (OntoPage). VIS functions can be propagated to other relevant data streams and links, a process that results in new OntoPage instances with the same VIS function and a new set of data streams and links. This is a novel aspect of our ontology-based approach to visualization, as existing VIS functions can be used to visualize numerous data streams, without any action from VIS developers.

Propagating a VIS function to generate plots and dashboards for other data streams is not straightforward. This requires actions from the infrastructure manager to ensure all appropriate data streams are correctly mapped and linked in the propagated visualizations. Whilst visual designs in our system are implemented by volunteer VIS developers, the infrastructure manager is a volunteer responsible for overall infrastructure management and visualization propagation. The infrastructure manager faces several challenges: (i) there are numerous data streams in the infrastructure and knowing which streams are available is difficult; (ii) some plots and dashboards have multiple data streams and links that need to be correctly matched, but searching for matching data streams and links is a group-based multi-criteria decision; (iii) when there are many possible matching results, quality-assurance is a mission-critical and demanding task.

Our propagation workflow (Fig. 2) has two tasks, carried out by the infrastructure manager: first, they need to formulate a query for data streams that can be part of a sensible binding with the chosen visualization; second, they must perform 'quality assurance' by reviewing search results, to determine whether to propagate the visualization and have it published. Our system has a search user interface to support these two tasks, which we now discuss separately.

### 6.1 User Interface for Search and Ranking

When a VIS function needs to be propagated, the infrastructure manager first needs to search for appropriate data streams. We have a significant number of data streams (e.g., dozens of metrics stratified by dozens of local authority regions) and reviewing every permutation of data streams for a new visualization will be time-consuming and impractical. The search process and user interface aims to help the infrastructure manager find good candidates for propagation. An effective search

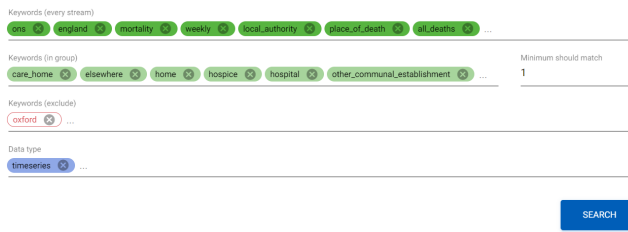


Fig. 5. Screenshot of the search user interface with an example query consisting of keywords for six data streams about weekly mortality rates in England. Keywords can be quickly added to a query via the search UI.

interface will help with quality assurance by reducing the potential for inappropriate bindings, reducing the volunteer time-cost. It will also help reduce the workload required to disseminate new visualizations, especially as the system scales with new data streams and more complex visualizations (e.g., dashboards with several plots).

Search and result ranking operates on keywords in the ontology. Since every VIS function is defined with reference data streams, our system extracts keywords from those references and uses them to search. When starting a new search for a chosen visualization function, the infrastructure manager is shown the keywords for the reference visualization and data streams (as in Fig. 5). There are four search bars for building a query: for identifying keywords that (1) must appear in every data stream, (2) must appear at least once within a group of results, (3) must not appear, and (4) for limiting data types.

Clicking on a keyword in the list of reference data streams will add it to the first search bar (i.e., must appear in every stream) and subsequent clicks will move them to the next bar, cycling through the three keyword criteria. Keywords are shown with a colored background, both in their original location and in the search bar (e.g., Fig. 5): dark green means the keyword is in every stream, pale green means a keyword must appear at least once in a group of results, and a red border and text means a keyword must be excluded. Keywords may be excluded because they are expected to vary or be omitted (e.g., when propagating a dashboard for one region to other regions). Data type filters are shown in their own search bar with a blue background.

Overall, this user interface supports query construction by using a visualization's reference data binding as the 'template' for building queries from ontology keywords. This reduces the need for text entry and ensures keywords are entered correctly. The user interface visualizes the search parameters *in situ* in the reference visualization by highlighting keywords in its data stream(s), helping the infrastructure manager verify the search criteria are formulated correctly. Search results are then presented in ranked order. Parameters for the ranking algorithm can also be adjusted via the UI if necessary, e.g., to specify the required number of matching keywords within a group of streams.

## 6.2 User Interface for Quality Assurance

Search results are presented with grouped and highlighted keywords so the infrastructure manager can see at a glance if a visualization function should be propagated to a set of results. This quality assurance is necessary to ensure that visualization functions are only propagated and published if they are appropriate for the underlying data. Importantly, this only happens once for each visualization function and data stream permutation: once a visualization is propagated and a new binding has been established, no further review is required. This allows the system to scale, without the need for frequent and time-consuming quality assurance. Downloader agents and propagation agents ensure visualizations on web pages are automatically kept up to date.

Having a human in the loop is vital as this is a complex decision process. The infrastructure manager needs to evaluate each search result. For a visualization for a single data stream, this is a straightforward check to decide if the data types match and the data would make sense for that visualization. For complex visualizations of multiple data streams (e.g., regional or national dashboards, multi-series plots), this requires more nuance. Data streams, types, and ontology keywords must be compared with the vis. function reference data streams, to



Fig. 6. Screenshot of the search results user interface for propagation quality assurance, showing three results with six data streams (for the Fig. 5 query). Keywords are grouped and highlighted by match type, so the infrastructure manager can glance at the results keywords and decide quickly if propagation should be activated (via the tick icon).

decide if propagation makes sense for each permutation of streams. Propagation only takes place once; i.e., if a visualization is propagated to a set of data streams, then that result is not shown in future.

Our search result user interface helps the infrastructure manager make these decisions using keyword grouping and highlighting. Whilst our search and ranking process reduces inappropriate matches, there is still a considerable amount of results to review. A good user interface thus has a large positive impact on reducing workload, which is important when dealing with data and visualizations of this nature.

Fig. 6 shows how search results are presented. Each result is a set of data streams that satisfy the query constraints. Each data stream is shown with its keywords, data type, and description. Keywords have the most significant influence on infrastructure manager's decisions about whether to propagate a visualization to a set of data streams, so we structured the keyword presentation to facilitate efficient comparison between the visualization reference data keywords and the result data stream keywords. If keywords appear in all reference and result streams they are not shown, since their presence is implicit from the query. Keywords that do not appear in the reference streams are shown first with a gray background to help identify differences. If keywords match a query term and appear in the correct order, they are shown with a pale green background. By grouping and presenting keywords like this, the infrastructure manager can decide if propagation should occur.

As an illustrated example, Fig. 6 shows three search results for a query (shown in Fig. 5) for a stacked bar chart with six data streams (shown in Fig. 1). Each result is a set of six data streams that meet the query constraints. The first column shows keywords that do not match the query terms, highlighted in gray. In the three search results, these unmatched keywords are for three other regions of England, so it is expected that they do not match query terms, as we want to propagate this design to other regions. In this example, the visualization shows data per region, so the infrastructure manager will see that these data streams are grouped by one region and valid for this visual design.

The second column shows keywords that matched query terms. In the first two results, all keywords in the second column are green, showing a complete match: these keywords appear in the query and correspond with the ordering in the visualization reference. Since these two results are correctly grouped by region and the data streams match, the infrastructure manager would choose to propagate (using the green check-mark icon). In the third result, all six keywords match the query



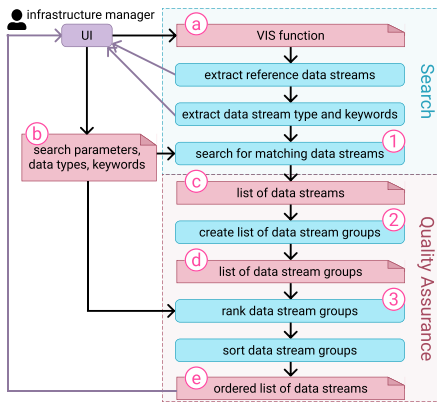


Fig. 7. A flowchart showing the search and ranking process. When the infrastructure manager selects a **VIS function (a)**, its reference data streams and their attributes are retrieved and shown in the search UI. They then construct a **query (b)** by clicking on, or manually entering, keywords then search. Search algorithms find matching **data streams (c)**, create **stream groups (d)**, sort them, then present an **ordered list (e)** through the results UI, highlighted to support fast visual scanning.

terms, but the last two appear in the incorrect place (and are highlighted gray to show this). Propagation should not take place for this result, as some data would appear incorrectly.

## 7 PROPAGATION TECHNICAL INFRASTRUCTURE

Our propagation process is supported by a technical infrastructure of search, grouping, and ranking algorithms. Fig. 7 shows the operations and algorithms underlying the search and quality assurance interfaces described in the previous section. When the infrastructure manager is ready to propagate a VIS function, its reference data stream keywords and metadata are extracted from the ontology.

Those reference data streams, keywords, and metadata are presented in the search UI (Section 6.1), to help the infrastructure manager construct their query. Keywords are assigned one of three categories: must appear in every stream, must appear at least once within a group of streams, or must be excluded. Additional keywords and search terms can also be provided via the search UI.

When the query is ready, a series of algorithms process all data streams in the ontology. There are three algorithms: **Searching** (Fig. 7–(1)), **Grouping** (Fig. 7–(2)), and **Ranking** (Fig. 7–(3)); the final sorting process is trivial based on ranks. By searching then sorting results in the quality assurance UI (Section 6.2), the infrastructure manager is shown the best candidates for propagation at the top of the search results.

In the following sections we discuss the Searching, Grouping, and Ranking algorithms. We focus on plot propagation as a simple example. When propagating a plot that consists of several data streams, the priority is to find matching data stream groups such that the semantic ordering of streams is correct (e.g., so that the correct categories of data appear in the same order). Propagating a dashboard is more complex, because web page links (i.e., OntoPage instances) also need to be grouped and ordered correctly. This is a more complicated process, which we describe in the supplementary material.

### 7.1 Data Stream Search

Keywords are important when searching for data streams, as they are used to identify similar data streams appropriate for the chosen visual design (e.g., the plot or dashboard being propagated). Once a query is constructed in the search UI, it is converted into the declarative query language DSL [13], for searching the database underlying the ontology.

As an illustrated example, consider Fig. 1. This shows stacked bar plots of regional weekly mortality data in England, which is split into six location types (care home, communal establishment, elsewhere, home, hospice, hospital) and provided for 336 regions. The Reference Visualization is for the Oxford region. Underlying this is an OntoPage instance, linked to the OntoVis instance for the VIS function and six OntoData instances for Oxford’s mortality data streams.

Suppose the infrastructure manager wishes to propagate this plot to the other 335 regions of England. They construct the query shown in Fig. 5 which specifies: keywords that must appear in every data stream (e.g., england, weekly, mortality, place\_of\_death, etc), keywords that should appear in at least one data stream in the group (i.e., keywords for each place of death); keywords that should be excluded (i.e., for the Oxford region). It also specifies stream data type (i.e., time series).

Let the reference data streams for the visualization function be  $R_1, R_2, R_3, \dots, R_k$  (where  $k = 6$  reference streams). We search for all data streams in the ontology that match the search criteria using the specified keywords. This results in a set of  $m$  discovered data streams  $D_{(i,1)}, D_{(i,2)}, \dots, D_{(i,m)}$ . In total, there are  $m = 335$  sets of data streams (and each set containing  $k$  data streams) discovered by our search algorithm, for all regions of England excluding Oxford.

### 7.2 Data Stream Grouping

The reference data streams for a visualization function form a group, where the order of data streams is important. Let the reference data streams be  $R_1, R_2, \dots, R_k$ ; we would like to create similar groups that match this. Discovered data streams (total  $n$ ) therefore need to be grouped in a similar way to the reference streams. In our example from Fig. 1, there are over 300 groups taken from thousands of data streams matching the query and, inevitably, there will be unwanted streams in the ontology that are discovered by the search algorithm.

Our grouping algorithm constructs groups from discovered data streams, aiming to maximize similarity with the reference stream group, outlined in Fig. 8. To do this, we compute two similarity matrices,  $S_{rd}$  and  $S_{dd}$ , which are of size  $k \times n$  and  $n \times n$  respectively. We compute a similarity measure  $\gamma(R_i, D_j)$  ( $i = 1..k, j = 1..n$ ) for each discovered data stream  $D_j$ . We compute another similarity measure  $\lambda(D_u, D_v)$  ( $u = 1..n, v = 1..n$ ) for each pair of discovered streams  $D_u$  and  $D_v$ .

The similarity functions  $\gamma()$  and  $\lambda()$  consider the similarity between data type, keywords, API endpoint and the description field. API endpoint and description similarity is computed using a text comparison algorithm, data type similarity is simple string matching function, and keyword similarity is based on comparison of sets. The three similarity measurement algorithms and computation of  $S_{rd}$  and  $S_{dd}$  are provided as Supplementary Material.

After computing the similarity matrices, the grouping algorithm examines the set of discovered data streams  $D_1, D_2, \dots, D_n$  and finds all permutations that meet a set of grouping requirements. Given a subset of data streams  $[D'_1, D'_2, \dots, D'_k] \subset [D_1, D_2, \dots, D_n]$ , the grouping requirements are defined using both similarity matrices  $S_{rd}$  and  $S_{dd}$ :

$$\begin{aligned} & \frac{1}{k} \sum_{i=1}^k \gamma(R_i, D'_i) > T_{\text{group}} \quad \wedge \quad \forall i = 1..k, \gamma(R_i, D'_i) > T_{\text{stream}} \\ & \wedge \quad \frac{2}{k(k-1)} \sum_{i=1}^{k-1} \sum_{j=i+1}^k \lambda(D'_i, D'_j) > S_{\text{allpair}} \\ & \wedge \quad \forall i = 1..k, j = 1..k, i \neq j, \lambda(D'_i, D'_j) > S_{\text{pair}} \end{aligned}$$

where  $T_{\text{group}}$ ,  $T_{\text{stream}}$ ,  $S_{\text{allpair}}$ , and  $S_{\text{pair}}$  are control parameters defined by the infrastructure manager. As a result, the grouping algorithm gives  $m$  groups  $G_1, G_2, \dots, G_m$ , each with  $k$  data streams.

**Grouping Data Streams.** In Section B of the supplementary material, we described two grouping algorithms, each with certain trade-offs. Algorithm 1 is based on a brute-force approach, which iterates through each row of the similarity matrix  $S_{dd}$  to find the  $k$  closest elements, and keeps iterating until all  $m$  groups are discovered. This works well in a situation where there exists exactly  $k$  closest elements in each row.

Algorithm 2 is based on a graph spectral method described in [24]. Graph spectral methods are applied to divide a graph’s closest vertices into equal size components. The similarity matrix  $S_{dd}$  can be seen as an adjacency matrix of a weighted undirected graph: each element of  $S_{dd}$  represents a node of the graph, and the similarity between any two elements is the weight of an edge between them. This algorithm takes  $S_{dd}$  as an input and returns  $m$  different groups  $G_i$ , and each group containing  $k$  data streams. This algorithm performs efficiently when the matrix is sparse and there exists only a small number of clusters.

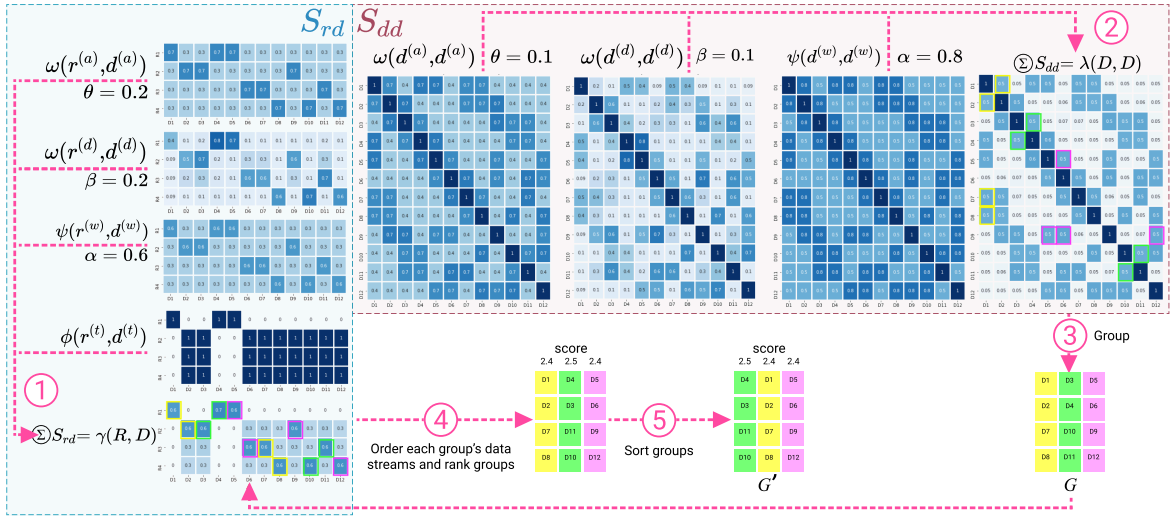


Fig. 8. Illustrates an example grouping, ordering and ranking workflow with four reference data streams ( $R_1, R_2, R_3, R_4$ ) and 12 discovered data streams ( $D_1, D_2 \dots D_{12}$ ). (1) We derive a similarity matrix  $S_{rd}$  measuring similarity between reference data streams and discovered data streams. First we derive feature vectors from reference data streams (e.g., API endpoint  $r^{(a)}$ , description  $r^{(d)}$ , keywords  $r^{(w)}$ , and data type  $r^{(t)}$ ) and discovered data streams (e.g., API endpoint  $d^{(a)}$ , description  $d^{(d)}$ , keywords  $d^{(w)}$ , and data type  $d^{(t)}$ ). Next, we compute a pairwise similarity matrix:  $\omega(r^{(a)}, d^{(a)})$ ,  $\omega(r^{(d)}, d^{(d)})$ ,  $\psi(r^{(w)}, d^{(w)})$ , and  $\phi(r^{(t)}, d^{(t)})$ . Finally, we aggregate by taking a weighted average of the matrices. (2) We derive the similarity matrix  $S_{dd}$  for discovered data streams using a similar process. (3) Grouping algorithm group similar data streams into uniform groups. (4) Data streams are ordered within each group, to match the reference stream order, then compute ranking scores. (5) Sort groups by ranking score.

The infrastructure manager can select which grouping algorithm to use when searching. Each performs better in different contexts and the decision to change will be ad hoc based on the results. For example, if there are a lot of data streams in the query then better results may be obtained by switching algorithm, whereas the graph spectral method will typically perform better for smaller numbers of groups.

**Ordering Data Streams.** In Section B of the supplementary material, we outline the process where streams within a group are ordered (Fig. 8—(4)). We iterate over each column of the derived matrix,  $G$ , to sort its streams, such that they match the reference group data streams based on the degree of similarity in similarity matrix  $S_{rd}$ .

### 7.3 Group Ranking

The ranking algorithm assigns a score to each group of data streams to indicate the likelihood that it may be suitable for propagation. The ranking score is computed based on both similarity matrices  $S_{rd}$  and  $S_{dd}$  described before. Given a group  $G_a$  ( $a = 1..m$ ) that contains  $k$  data streams  $[D_1^a, D_2^a, \dots, D_k^a]$ , the ranking score is defined as:

$$S(G_a) = \frac{W}{k} \left( \sum_{i=1}^k \gamma(R_i, D_i^a) \right) + \frac{2(1-W)}{k(k-1)} \left( \sum_{i=1}^{k-1} \sum_{j=i+1}^k \lambda(D_i^a, D_j^a) \right)$$

where  $W$  is a control parameter in the range of  $[0, 1]$  for controlling the contributions of the two types of similarity measures. Once a ranking score is assigned to each group, sorting them is trivial to determine the presentation order. The  $m$  sorted groups, each with  $k$  data streams, are sent to the results UI for quality assurance (Section 6.2).

## 8 EVALUATION

We conducted a qualitative evaluation of the propagation interface and workflow during the development process, to reflect on our design and formatively evaluate the workflow effectiveness. There were six participants: two visualization researchers (our infrastructure and data managers respectively, co-authors), two experienced analytics developers (Power BI, Salesforce Einstein Analytics, and Tableau), one software developer, and one postgraduate student specializing in visualization. Each session began with a tutorial, after which participants were asked to complete two propagation tasks using real COVID-19 data and existing visualization functions. The session ended with a semi-structured interview. Each session lasted 60–90 minutes.

We asked participants to reflect on the overall process of propagating plots to new data streams. As visualization researchers and developers,

all understood the problem that propagation addresses. All participants recognised the time that propagation could save: some noted the time needed to search and group sets of data streams manually then integrate into new visualizations. Others highlighted the risk of costly errors in a manual process and said that the grouped and ranked results meant propagation was about “sanity checking” rather than making complex decisions. None of them had seen propagation-like features in other visualization platforms or tools, whilst those with industry experience suggested that the platforms they use, and their own working practice, could benefit from such features.

We also discussed the search and results user interface designs. When constructing queries, all used the quick keyword selection feature rather than type keywords; some said during the interview that this helped them create their search terms more quickly and meant they would not need to memorize keywords. The keyword colour coding in the search form seemed intuitive, although was most useful to participants when viewing the search results. Colour-coded keywords helped them decide which results to propagate visualization functions to, with users finding the keyword grouping to be especially useful; this meant they could scan and identify suitable groups, such that they could complete the tasks with less time and cognitive demand.

## 9 DISCUSSION

This work was motivated by the significant need for visual analytics to support the emergency response to COVID-19. A significant volume and diversity of visual designs were required to support epidemiologists, modeling scientists and other domain experts in the SCRC, but we needed an approach that was feasible for a team of VIS volunteers in a context where timeliness was critical. As discussed in Section 3, we considered a number of solutions such as using existing visualization platforms, but the need for bespoke visualization and dashboard designs, and for strategically efficient use of volunteer resources, led us to the streamlined development and propagation approach outlined here.

### 9.1 What did We Achieve?

We observed, during our ongoing research project, a number of notable benefits of our method. We were able to: (i) **re-purpose and reuse a given visualization** design in various contexts by propagating across numerous data streams, both for individual plots and composite dashboards, in an efficient yet controlled manner, thereby responding to the need for volume and diversity in visualizations; (ii) ensure the



suitability and efficacy of the visualizations offered through a **semi-automatic propagation process** that facilitates visualization quality assurance; (iii) **streamline the visualization development** process by separating visualization development from infrastructure management; and (iv) strategically **target our limited volunteer and development resources** to where they can make the most impact in a short time. We open up these points to further discussion in the following.

When visualizations are propagated across numerous different data streams or re-purposed within various dashboards, **quality assurance is of paramount importance**; visualizations need to be checked to ensure data streams (via their data types and keywords in our ontology) are suitable for the given plot and/or dashboard. This is especially important in the case of a visualization system developed to support the response to the pandemic, since the visualizations are involved in critical inference and decision-making scenarios. One can argue that automated visualization generation [41] could be an alternative to streamline the visualization propagation process, but we have observed in our solution that a fully automated approach is not always reliable and can lead to unsuitable propagation. Given the importance of this task and such potential limitations in algorithmic methods, we developed a semi-automated approach that uses an ontology with algorithmic support for searching and ranking data streams for propagation, with a user interface that supports a infrastructure manager in assessing and approving the recommendations from the algorithms.

## 9.2 Importance of Roles and Separation of Concerns

Throughout this project (and in our ongoing efforts), we were faced with a growing need for plots and dashboards for a range of data sources, while the resources for developing these were limited. Our approach addresses this in two ways. First, by limiting demand on VIS developer time by propagating a visual design to all datasets that will be useful to the domain experts. Second, by enabling a more strategic approach to resource management by **decoupling visualization design from data management**. This keeps visualization developers away from the complexities of the infrastructure and gives them more time to focus on designing and developing novel visualization capability. Meanwhile, the data infrastructure and propagation workflows are managed by dedicated developers who are well-versed in the data streams and infrastructure, and are skilled at ensuring the quality of the propagation via quality assurance. These volunteers are not responsible for VIS design. This **separation of roles** is not only an effective use of developer time, but ensures the integrity of the final product. While we present these roles as distinct individuals in the paper, in reality, there may be overlaps and the same individual might be wearing multiple hats, e.g., a developer who is comfortable in designing and developing visualizations could also contribute to data management.

**Our template-based propagation framework enables the separation of visualization design and implementation.** Following on from the growing trend of visualization specification languages [30], this enables the design of plots to be explicitly specified and formulated without the constraints of the data infrastructure. This has a number of advantages, in that it is easier to ensure consistency across numerous plots and dashboards, and provides **consistency in how visualizations are presented over the web**, e.g., consistency in naming, titles, details of descriptions. Such an approach also makes the designs more transferable to contexts where new data streams and visual analytic needs may arise in short time. For instance, with the introduction of vaccination, it is possible to propagate several existing visualizations used for case/hospitalization numbers to this new context.

## 9.3 Upfront Costs

Our approach to designing and developing a propagation mechanism did take significant time and resources to get the system in operation. The **implementation of the data and propagation infrastructure was a significant development effort** and, for a while, most of the development had to happen in the “backstage,” with limited progress to demonstrate to domain experts in terms of visualization selection. However, our novel approach is a result of overcoming these technical challenges. Once the propagation system was functional, our approach

was able to multiply the designs to various context and enable the rapid deployment a wide portfolio of visualizations. **While initial progress in terms of visualization offering would be slow**, our scalable and flexible approach “future-proofs” the system as new data products become available, to meet the varying VIS needs of domain experts.

## 9.4 Generalizing Our Approach

While our approach has been motivated by the ongoing pandemic, the proposed propagation approach and the workflow that our framework supports is transferable to different data-intensive settings, where there is demand for diverse and abundant visualization designs with large collections of data streams. The ontology-supported approach and underlying schema can be generalized and adapted to new data contexts. Our ontology models the data and VIS infrastructure, but **domain knowledge exists via data stream attributes** (i.e., keywords, descriptions), decoupled from the infrastructure implementation. Our system can rapidly transfer to a new domain by adding new data streams and capturing domain knowledge in their attributes, while a set of generic **visualizations (plots and dashboards) would be already available**. One potential benefit in such a “transfer” would be the ability to **propagate the visualization designs** and re-use them in suitable combinations within dashboards tailored for the specifics of the new application context. The transferability is also key for ensuring the preparedness of the visualization response in future situations where time-critical VIS is essential and could provide a solid foundation for further development.

## 10 CONCLUSION

This paper presents an ontology-based visualization development and propagation framework, with a streamlined workflow developed in response to the significant development challenges faced by the RAMPVIS volunteer visualization effort while responding to the COVID-19 pandemic. Our key challenge was to meet the need for a large number of diverse plots and dashboards, to meet the constantly evolving visual analytic requirements of domain experts in the Scottish COVID-19 Response Consortium. Meeting this challenge with scarce development and volunteer resources was only possible through a carefully designed infrastructure that streamlines the development process.

We do this through a visual design workflow that separates VIS development from the data infrastructure. Our ontology plays a key role in our infrastructure, capturing the relationships between data streams, VIS functions and web pages. We used an ontology-supported propagation process to allow a particular visualization to be rapidly deployed across numerous suitable data streams, instantly deploying them as interactive web pages. This enables a workflow that allows VIS volunteers focus their efforts on tasks they are most effective in.

Our approach now enables the RAMPVIS consortium to offer a wide range of quality-assured plots and dashboards within a consistent presentation framework. With the changing demands of the ongoing pandemic management efforts (e.g., attention shifting to vaccination campaigns), our approach makes the visualization response from the consortium more resilient, responsive, and sustainable. We are currently working closely together with SCRC to adapt our system to the rapidly changing nature of the pandemic and to improve our visualizations, dashboards, and user interfaces for use through the domain experts. In conclusion, we argue that our approach could serve as a blueprint for similar volunteer VIS efforts in future. In situations where the timely delivery of large-scale visualization is mission-critical, frameworks like these strengthen the key role that visualization plays in informing critical inference and decision-making.

## ACKNOWLEDGMENTS

This work was supported by EPSRC (EP/V054236/1). We would like to thank all volunteers from the SCRC and all VIS volunteers [3]. We would also like to thank Prof. N. W. John (U. Chester) and Dr H. C. Purchase (U. Glasgow) for their involvement in work of the generic support team. We are grateful to Dr R. Reeve (U. Glasgow) and A. Brett (UKAEA) for their leadership in creating the SCRC data infrastructure that the VIS infrastructure depends on, and A. Lahiff and his STFC colleagues for maintaining the RAMP VIS VMs, and S. Michell (U. Glasgow) for offering valuable advice on data products.

## REFERENCES

- [1] D3.js: Data-Driven Documents. <https://d3js.org/>. Accessed on 31 March 2021.
- [2] Jinja Template Documentation. <https://jinja.palletsprojects.com/en/2.11.x/templates>. Accessed on 31 March 2021.
- [3] RAMPVIS Volunteers. <https://sites.google.com/view/rampvis/volunteers>. Accessed on 31 March 2021.
- [4] Apache Software Foundation. Lucene. <https://lucene.apache.org>, 2021. Accessed on 15 March 2021.
- [5] M. Bostock, V. Ogievetsky, and J. Heer. D<sup>3</sup> data-driven documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, 2011. doi: 10.1109/TVCG.2011.185
- [6] K. Brodlie, J. Brooke, M. Chen, D. Chisnall, A. Fewings, C. Hughes, N. John, M. Jones, M. Riding, and N. Roard. Visual supercomputing – Technologies, applications and challenges. *Computer Graphics Forum*, 24(2), 2005.
- [7] S. Carpendale, M. Chen, D. Evanko, N. Gehlenborg, C. Görg, L. Hunter, F. Rowland, M.-A. Storey, and H. Strobel. Ontologies in biological data visualization. *IEEE Computer Graphics and Applications*, 34(2):8–15, 2014.
- [8] M. Chen, A. Abdul-Rahman, D. Archambault, J. Dykes, A. Slingsby, P. D. Ritsos, T. Torsney-Weir, C. Turkay, B. Bach, A. Brett, H. Fang, R. Jianu, S. Khan, R. S. Laramée, P. H. Nguyen, R. Reeve, J. C. Roberts, F. Vidal, Q. Wang, J. Wood, and K. Xu. RAMPVIS: Towards a new methodology for developing visualisation capabilities for large-scale emergency responses. arXiv:2012.04757, 2020.
- [9] M. Chen, G. Grinstein, C. R. Johnson, J. Kennedy, and M. Tory. Pathways for theoretical advances in visualization. *IEEE Computer Graphics and Applications*, 37(4):103–112, 2017. doi: 10.1109/MCG.2017.3271463
- [10] Y. Chen, S. Barlowe, and J. Yang. Click2annotate: Automated insight externalization with rich semantics. In *2010 IEEE Symposium on Visual Analytics Science and Technology*, pp. 155–162, 2010. doi: 10.1109/VAST.2010.5652885
- [11] W. Cui, X. Zhang, Y. Wang, H. Huang, B. Chen, L. Fang, H. Zhang, J. G. Lou, and D. Zhang. Text-to-viz: Automatic generation of infographics from proportion-related natural language statements. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):906–916, 2020. doi: 10.1109/TVCG.2019.2934785
- [12] M. Dudáš, S. Lohmann, V. Svátek, and D. Pavlov. Ontology visualization methods and tools: a survey of the state of the art. *The Knowledge Engineering Review*, 33, 2018.
- [13] Elastic NV. Query DSL. <https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl.html>, 2021. Accessed on 17 March 2021.
- [14] S. M. Falconer, R. I. Bull, L. Grammel, and M. Storey. Creating visualizations through ontology mapping. In *2009 International Conference on Complex, Intelligent and Software Intensive Systems*, pp. 688–693, 2009. doi: 10.1109/CISIS.2009.40
- [15] O. Gilson, N. Silva, P. W. Grant, and M. Chen. From web data to visualization via ontology mapping. In *Proceedings of the 10th Joint Eurographics / IEEE - VGTC Conference on Visualization*, pp. 959–966, 2008. doi: 10.1111/j.1467-8659.2008.01230.x
- [16] L. Grammel, C. Bennett, M. Tory, and M.-A. Storey. A Survey of Visualization Construction User Interfaces. In M. Hlawitschka and T. Weinkauff, eds., *EuroVis - Short Papers*, 2013. doi: 10.2312/PE.EuroVisShort.EuroVisShort2013.019-023
- [17] P. Jaccard. The distribution of the flora in the alpine zone. *New Phytologist*, 11(2):37–50, 1912. doi: 10.1111/j.1469-8137.1912.tb05611.x
- [18] A. Katifori, C. Halatsis, G. Lepouras, C. Vassilakis, and E. Giannopoulou. Ontology visualization methods—a survey. *ACM Comput. Surv.*, 39(4):10–es, Nov. 2007. doi: 10.1145/1287620.1287621
- [19] J. O. Kephart and D. M. Chess. The vision of autonomic computing. *Computer*, 36(1):41–50, 2003.
- [20] S. Khan, A. Abdul-Rahman, P. H. Nguyen, and B. Bach. RAMPVIS ui. <https://github.com/ScottishCovidResponse/rampvis-ui>. Accessed on 31 March 2021.
- [21] S. Khan, U. Kanturska, T. Waters, J. Eaton, R. Bañares Alcántara, and M. Chen. Ontology-assisted provenance visualization for supporting enterprise search of engineering and business files. *Advanced Engineering Informatics*, 30(2):244–257, Apr. 2016. doi: 10.1016/j.aei.2016.04.003
- [22] S. Khan and P. H. Nguyen. RAMPVIS api. <https://github.com/ScottishCovidResponse/rampvis-api>. Accessed on 31 March 2021.
- [23] S. Khan and P. H. Nguyen. RAMPVIS ontology management and propagation UI. <https://github.com/saifulkhan/rampvis-ontology-management-ui>. Accessed on 31 March 2021.
- [24] J. Leskovec, A. Rajaraman, and J. D. Ullman. *Mining of Massive Datasets*. Cambridge University Press, 2020.
- [25] J. Mackinlay. Automating the design of graphical presentations of relational information. *ACM Transactions on Graphics*, 5(2):110–141, Apr. 1986. doi: 10.1145/22949.22950
- [26] J. Mackinlay, P. Hanrahan, and C. Stolte. Show me: Automatic presentation for visual analysis. *IEEE Transactions on Visualization and Computer Graphics*, 13(6):1137–1144, Nov. 2007. doi: 10.1109/TVCG.2007.70594
- [27] C. D. Manning, P. Raghavan, and H. Schütze. *An Introduction to Information Retrieval*. Cambridge University Press, 2008. doi: 10.1109/LPT.2009.2020494
- [28] D. Ren, B. Lee, and M. Brehmer. Charticator: Interactive construction of bespoke chart layouts. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):789–799, 2019. doi: 10.1109/TVCG.2018.2865158
- [29] A. Satyanarayan and J. Heer. Lyra: An interactive visualization design environment. *Computer Graphics Forum*, 33(3):351–360, 2014. doi: 10.1111/cgf.12391
- [30] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, 2017. doi: 10.1109/TVCG.2016.2599030
- [31] Scottish Covid Response Consortium. SCRC Data Registration and Management System. [data.scrc.uk](https://data.scrc.uk), 2021. Accessed on 31 March 2021.
- [32] D. Shi, X. Xu, F. Sun, Y. Shi, and N. Cao. Calliope: Automatic visual data story generation from a spreadsheet. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):453–463, 2021. doi: 10.1109/TVCG.2020.3030403
- [33] Y. Sun, J. Leigh, A. Johnson, and S. Lee. Articulate: A semi-automated model for translating natural language queries into meaningful visualizations. In *Proceedings of the 10th International Conference on Smart Graphics*, pp. 184–195. Springer-Verlag, Berlin, Heidelberg, 2010.
- [34] T. Tang, R. Li, X. Wu, S. Liu, J. Knittel, S. Koch, T. Ertl, L. Yu, P. Ren, and Y. Wu. PlotThread: Creating expressive storyline visualizations using reinforcement learning. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):294–303, 2021. doi: 10.1109/TVCG.2020.3030467
- [35] Tanimoto T.T. An Elementary Mathematical theory of Classification and Prediction. Technical report, IBM Corporation, 1958.
- [36] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer. Voyager: Exploratory analysis via faceted browsing of visualization recommendations. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):649–658, 2016. doi: 10.1109/TVCG.2015.2467191
- [37] A. Wu, Y. Wang, X. Shu, D. Moritz, W. Cui, H. Zhang, D. Zhang, and H. Qu. Survey on artificial intelligence approaches for visualization data, 2021. Pre-print: arXiv cs.HC 2102.01330.
- [38] B. Yu and C. T. Silva. VisFlow - Web-based visualization framework for tabular data with a subset flow model. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):251–260, 2016.
- [39] B. Yu and C. T. Silva. FlowSense: A natural language interface for visual data exploration within a dataflow system. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):1–11, 2019.
- [40] R. Zehring, A. Singhal, M. Correll, and L. Battle. Vis Ex Machina: An analysis of trust in human versus algorithmically generated visualization recommendations, 2021. Pre-print: arXiv cs.HC 2101.04251.
- [41] S. Zhu, G. Sun, Q. Jiang, M. Zha, and R. Liang. A survey on automatic infographics and visualization recommendations. *Visual Informatics*, 4(3):24–40, 2020. doi: 10.1016/j.visinf.2020.07.002